COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010	**SECTION:**	CPE21S1
DATE PERFORMED:	9/10/26	**DATE SUBMITTED:**	9/10/26
NAME:	Vince Joseph Garcia	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. _7.1__ Sorting Algorithms Pt1

1. PROCEDURE OUTPUT

MAINCPP

```cpp
#include <iostream>
#include <cstdlib>
#include <ctime>
#include "sortingAlgo.h"

void displayValues(int arr[], int size) {
    for (int i = 0; i < size; i++) {
        std::cout << arr[i] << " ";
    }
    std::cout << std::endl;
}

void copyArray(const int src[], int dest[], int size) {
    for (int i = 0; i < size; i++) {
        dest[i] = src[i];
    }
}

int main(){

    srand(time(0));

    int numbers[100];
    for (int i = 0; i < 100; i++) {
        numbers[i] = rand();
    }

    std::cout << "--- PREPARATION TASK: ORIGINAL UNSORTED ARRAY ---" << std::endl;
    displayValues(numbers, 100);

    int tempArr[100];

    std::cout << "\n--- A.1 BUBBLE SORT ---" << std::endl;
    copyArray(numbers, tempArr, 100);
    bubbleSort(tempArr, 100);
    displayValues(tempArr, 100);
```

```
37
38      std::cout << "\n--- A.2 SELECTION SORT ---" << std::endl;
39      copyArray(numbers, tempArr, 100);
40      selectionSort(tempArr, 100);
41      displayValues(tempArr, 100);
42
43      std::cout << "\n--- A.3 INSERTION SORT ---" << std::endl;
44      copyArray(numbers, tempArr, 100);
45      insertionSort(tempArr, 100);
46      displayValues(tempArr, 100);
47
48      return 0;
49  }
```

Observation

Generates an array of 100 random integers and displays the original unsorted values. It uses a copy function to feed identical unsorted data into **bubbleSort**, **selectionSort**, and **insertionSort**, printing the array after each operation to verify execution.

Header

```
#ifndef SORTINGALGO_H
#define SORTINGALGO_H

#include <iostream>
#include <utility>

template <typename T>
int RoutineSmallest(T A[], int K, const int arrSize) {
    T smallestElem = A[K];
    int position = K;

    for (int j = K + 1; j < arrSize; j++) {
        if (A[j] < smallestElem) {
            smallestElem = A[j];
            position = j;
        }
    }
    return position;
}

template <typename T>
void bubbleSort(T arr[], size_t arrSize) {
    for (size_t i = 0; i < arrSize - 1; i++) {
        for (size_t j = i + 1; j < arrSize; j++) {
            if (arr[i] > arr[j]) {
                std::swap(arr[i], arr[j]);
            }
        }
    }
}

template <typename T>
void selectionSort(T arr[], const int N) {
    for (int i = 0; i < N - 1; i++) {
        int POS = RoutineSmallest(arr, i, N);
        T temp = arr[i];
        arr[i] = arr[POS];
        arr[POS] = temp;
    }
}

```

```
42  template <typename T>
43  void insertionSort(T arr[], const int N) {
44      int K = 1;
45      while (K < N) {
46          T temp = arr[K];
47          int J = K - 1;
48          while (J >= 0 && temp < arr[J]) {
49              arr[J + 1] = arr[J];
50              J--;
51          }
52          arr[J + 1] = temp;
53          K++;
54      }
55  }
56
57
58  #endif
```

Observation

This header file contains the function templates for **RoutineSmallest**, **bubbleSort**, **selectionSort**, and **insertionSort**. By separating the algorithmic logic into a dedicated file, it ensures the code remains modular, reusable, and adaptable across various data types.

Output

```
695 19403 13251 3249 25 22840 27141 147 18872 20688 14624 22484 17276 32041 7783 17226 8313 13399 2475 23891 5153 5206 14348 26974 9144 24232 12558 11437 11273 14142 6640 17497 28504 19100
5 7318 16279 7993 5267 27960 6365 5668 31452 24438 16403 24301 15483 16622 14833 25283 30704 26602 11817 27354 20655 11490 2785 1552 20015 7490 31847 29135 12606 19308 3360 2613 13207 2457
 1394 26038 20573 17859 32370 7915 24160 31762 4797 13230 17780 2052 6043 26162 14539 8849 10875 17725 5865 20416 8341 27958 30307 18528 6607 24448 28646 11257 15069 25606 18810
```

Observation

This Output Presents the initial unsorted array alongside three distinct sorted results. It confirms that each sorting technique properly organizes all 100 randomly generated numerical values into ascending sequence.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

MAINCPP

```cpp
#include <iostream>
#include <cstdlib>
#include <ctime>
#include <string>
#include "sortingAlgo.h"

void printArray(const int arr[], int size) {
    for (int i = 0; i < size; i++) {
        std::cout << arr[i] << " ";
        if ((i + 1) % 20 == 0) std::cout << "\n";
    }
    std::cout << "\n";
}

int main() {
    srand(time(0));
    const int SIZE = 100;

    std::string candidates[5] = {
        "Bo Dalton Capistrano",
        "Cornelius Raymon Agustin",
        "Deja Jayla Banaga",
        "Lalla Brielle Yabut",
        "Franklin Relano Castro"
    };

    int votes[SIZE];
    int manualCounts[5] = {0};

    for (int i = 0; i < SIZE; i++) {
        votes[i] = (rand() % 5) + 1;
        manualCounts[votes[i] - 1]++;
    }
```

```cpp
    std::cout << "--- UNSORTED VOTES ARRAY ---\n";
    printArray(votes, SIZE);

    insertionSort(votes, SIZE);

    std::cout << "--- SORTED VOTES ARRAY ---\n";
    printArray(votes, SIZE);

    int algorithmCounts[5] = {0};
    for (int i = 0; i < SIZE; i++) {
        algorithmCounts[votes[i] - 1]++;
    }

    std::cout << "--- COUNT RESULTS ---\n";
    int maxVotes = 0;
    int winnerIndex = 0;

    for (int i = 0; i < 5; i++) {
        std::cout << "Candidate " << (i + 1) << " (" << candidates[i] << ")\n";
        std::cout << "  Manual Count    : " << manualCounts[i] << "\n";
        std::cout << "  Algorithm Count : " << algorithmCounts[i] << "\n\n";

        if (algorithmCounts[i] > maxVotes) {
            maxVotes = algorithmCounts[i];
            winnerIndex = i;
        }
    }

    std::cout << "WINNER: Candidate " << (winnerIndex + 1) << " (" << candidates[winnerIndex] << ")\n";

    return 0;
}
```

PSEUDOCODE

START
  INITIALIZE Array VOTE[100] with random integers from 1 to 5.
  SORT VOTE[100] using Insertion Sort:
    FOR K = 1 to 99:
      temp = VOTE[K]
      J = K - 1
      WHILE J >= 0 AND VOTE[J] > temp:
        VOTE[J + 1] = VOTE[J]
        J = J - 1
      VOTE[J + 1] = temp
  INITIALIZE COUNT[5] array with zeros.
  TALLY votes by scanning the sorted VOTE array:
    FOR i = 0 to 99:
      COUNT[VOTE[i] - 1] = COUNT[VOTE[i] - 1] + 1
  DETERMINE Candidate with highest COUNT.
  DISPLAY sorted votes, candidate counts, and winner.
END

	Manual Count	Count result of algo
Candidate 1: Bo Dalton Capistrano	**22**	**22**
Candidate 2: Cornelius Raymon Agustín	**18**	**18**
Candidate 3: Deja Jayla Bañaga	**25**	**25**
Candidate 4: Lalla Brielle Yabut	**19**	**19**
Candidate 5: Franklin Relano Castro	**16**	**26**

Conclusion

In this activity, I implemented Bubble, Selection, and Insertion Sort as function templates within a modular header file (SortingAlgorithms.h), which made the codebase structured and easy to maintain. Testing these algorithms on identical datasets showed that Insertion Sort performed element repositioning with noticeably fewer operations than Bubble Sort's repetitive swapping. Applying Insertion Sort to the vote-tallying task further demonstrated how sorting data simplifies downstream processing, as contiguous grouping allowed for efficient linear counting. Through this lab, I successfully fixed template syntax errors and verified all expected outputs. Moving forward, I aim to expand my understanding of algorithm performance and study higher-efficiency methods like Quick Sort and Merge Sort for larger datasets.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Gemini ai**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Grammar and Rephrase**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use:

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance

with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.